

MAXIMUM AVERAGE PASS RATIO

Company: Amazon

Round: Online Assessment

Year: 2021

Difficulty: Medium

Topic: Greedy, Priority Queue, Max Heap

Source: <https://leetcode.com/problems/maximum-average-pass-ratio>

Problem Description:

There are several classes in a school.

For every class, we are given two numbers : [pass, total]

- pass = number of students who will pass.
- total = total number of students in the class.

We are also given some extra students.

These extra students are special because they are guaranteed to pass, no matter which class we put them in.

Our task is to put these extra students into the classes in the best possible way so that the average pass ratio of all classes becomes as large as possible.

The pass ratio of a class is:

$\text{pass} / \text{total}$

Input Format:

The input contains:

- A 2D integer array classes
- An integer extraStudents

Output Format:

Return the maximum possible average pass ratio after placing all the extra students. The answer should be accurate up to 10^{-5} .

Sample Input:

classes = [[1,2],[3,5],[2,2]]

extraStudents = 2

Sample Output:

Maximum Possible Average Pass Ratio = 0.78333

Explanation:

Let's understand the first example in a simple way.

We have:

classes = [[1,2],[3,5],[2,2]]

extraStudents = 2

So the three classes are:

Class 1 → 1 out of 2 students pass

Class 2 → 3 out of 5 students pass

Class 3 → 2 out of 2 students pass

Their pass ratios are:

Class 1 → $1/2 = 0.5$

Class 2 → $3/5 = 0.6$

Class 3 → $2/2 = 1.0$

We have 2 extra students.

The important question is:

Which class should receive the next extra student?

We should not simply choose the class with the lowest ratio.

Instead, we should check:

Which class gets the biggest improvement if we add one student?

First extra student

If we give the student to Class 1:

$[1,2] \rightarrow [2,3]$

Its ratio changes from:

$1/2 = 0.5$

to:

$2/3 = 0.6667$

That's a big improvement.

If we give the student to Class 2:

$[3,5] \rightarrow [4,6]$

Its ratio changes from:

$3/5 = 0.6$

to:

$4/6 = 0.6667$

This is a smaller improvement.

Class 3 already has:

$$2/2 = 1$$

so adding a student would not improve its ratio.
Therefore, the first student should go to Class 1.

Second extra student

After the first student, Class 1 becomes:
[2,3]

We again check which class will benefit the most from the second student.
Class 1 still gives the best improvement, so we give the second student to Class 1 as well:
[2,3] $\rightarrow$ [3,4]

Now the classes are:
[3,4]

[3,5]

[2,2]

Their ratios are:
 $3/4 = 0.75$

$$3/5 = 0.60$$

$$2/2 = 1.00$$

So the average is:
 $30.75 + 0.60 + 1.00$

which gives:
0.78333

Therefore, the answer is:
0.78333

Approach to Solve This Problem:

The easiest way to think about this problem is:

Always give the next extra student to the class that gets the biggest improvement from that student.

To do this efficiently, we use a Priority Queue (Max Heap).

Step 1: Find the improvement for each class

For every class, we calculate how much its pass ratio will increase if we add one extra student.

Suppose we have:

[1,2]

Current ratio:

$$1/2 = 0.5$$

After adding one student:

[2,3]

New ratio:

$$2/3 = 0.6667$$

So the improvement is:

$$0.6667 - 0.5 = 0.1667$$

We call this improvement the gain.

Step 2: Put the classes in a Max Heap

A Max Heap keeps the class with the largest gain at the top.

This helps us quickly find the class that should receive the next extra student.

So instead of checking every class again and again, we simply take the class at the top.

Step 3: Give one extra student to the best class

We take the class with the largest gain.

For example:

[1,2]

becomes:

[2,3]

because the extra student is guaranteed to pass.

Step 4: Calculate its new gain

After adding the student, the class has changed.

So its gain may also change.

We calculate its gain again and put the updated class back into the Max Heap.

Step 5: Repeat

We repeat the same process for every extra student:

Find best class



Give it one student



Update the class



Calculate new gain



Put it back in the heap

After all extra students are assigned, we calculate the final average.

Java Code:

```
import java.util.*;

public class Main {

    // Calculates how much the pass ratio improves
    // after adding one extra student
    public static double gain(int pass, int total) {

        double oldRatio = (double) pass / total;
        double newRatio = (double) (pass + 1) / (total + 1);

        return newRatio - oldRatio;
    }

    public static double maxAverageRatio(int[][] classes, int extraStudents) {

        // Max Heap: class with the highest gain comes first
        PriorityQueue<int[]> pq = new PriorityQueue<>((
            (a, b) -> Double.compare(
                gain(b[0], b[1]),
                gain(a[0], a[1])
            )
        ));

        // Add all classes to the priority queue
        for (int[] c : classes) {
            pq.offer(c);
        }
    }
}
```

```
// Assign extra students one by one
for (int i = 0; i < extraStudents; i++) {

    // Get the class with the biggest gain
    int[] current = pq.poll();

    // Add one extra student
    current[0]++;
    current[1]++;

    // Put the updated class back
    pq.offer(current);
}

// Calculate the final average pass ratio
double totalRatio = 0;

while (!pq.isEmpty()) {

    int[] current = pq.poll();

    totalRatio += (double) current[0] / current[1];
}

return totalRatio / classes.length;
}
```

```
Run | Debug
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    // Number of classes
    int n = sc.nextInt();

    int[][] classes = new int[n][2];

    // Enter pass and total students
    for (int i = 0; i < n; i++) {
        classes[i][0] = sc.nextInt();
        classes[i][1] = sc.nextInt();
    }

    // Number of extra students
    int extraStudents = sc.nextInt();

    double answer = maxAverageRatio(classes, extraStudents);

    System.out.printf(format: "%.5f\n", answer);

    sc.close();
}
```

Solution:

The code follows three simple steps:

1. Calculate the gain for every class. Gain tells us how much the pass ratio will improve if we add one student.
2. Use a Max Heap to always find the class with the biggest gain.
3. Give the extra students one by one, update the selected class, and put it back into the heap.

After all extra students have been assigned, we calculate the pass ratio of every class and find their average. The main idea is to always choose the class where the next extra student will make the biggest difference. This greedy approach gives us the maximum possible average pass ratio.

Understanding the Important Parts of Code:

1. gain() function

```
public static double gain(int pass, int total) {  
    double oldRatio = (double) pass / total;  
    double newRatio = (double) (pass + 1) / (total + 1);  
  
    return newRatio - oldRatio;  
}
```

This finds how much the pass ratio improves when we add one extra student.

- oldRatio → ratio before adding the student
- newRatio → ratio after adding the student
- gain → improvement in the ratio

2. Max Heap

```
PriorityQueue<int[]> pq = new PriorityQueue<>(  
    (a, b) -> Double.compare(  
        gain(b[0], b[1]),  
        gain(a[0], a[1])  
    )  
);
```

We use a Priority Queue (Max Heap) to keep the class with the highest gain at the top.

This helps us quickly find which class should get the next extra student.

3. Get the Best Class

```
int[] current = pq.poll();  
poll() gives us the class with the highest gain.
```

That is the class where the next student will improve the pass ratio the most.

4. Add One Student

```
current[0]++;  
current[1]++;
```

Both values increase by 1.

For example:

$[1, 2] \rightarrow [2, 3]$

The extra student is guaranteed to pass, so both the number of passing students and total students increase.

5. Put the Class Back

```
pq.offer(current);
```

After adding the student, the class's gain changes.

So we put it back into the heap with its updated values.

6. Repeat

```
for (int i = 0; i < extraStudents; i++)
```

We repeat the same process for every extra student:

Find best class



Add student



Update class



Put it back

7. Calculate Final Average

```
totalRatio += (double) current[0] / current[1];
```

We calculate the final ratio of every class.

Then:

```
return totalRatio / classes.length; gives the average pass ratio.
```


Complexity Analysis

Time Complexity: $O((n + k) \log n)$

First, we put all n classes into the priority queue.

Then, for every extra student, we remove one class and put the updated class back.

Each priority queue operation takes $O(\log n)$.

Therefore : $O((n + k) \log n)$

Space Complexity: $O(n)$

The priority queue stores all the classes.

Therefore : $O(n)$